

Análise de Desempenho de Servidores RTMP e Transcodificadores no Streaming de Vídeo

Orientado: Valter Abreu Silva Junior

Departamento de Informática – Universidade Federal do Maranhão (UFMA)

Av. dos Portugueses, 1966 – Cidade Universitária Dom Delgado

65080-805 – São Luís – MA – Brasil

Orientador: Mario Antonio Meireles Teixeira

[Preencher: Cidade – ano]

Resumo

Este trabalho apresenta um estudo experimental para avaliar o desempenho de servidores RTMP e transcodificadores empregados em uma plataforma de streaming de vídeo ao vivo. A plataforma implementada combina Nginx-RTMP ou SRS para ingestão, FFmpeg ou GStreamer para transcodificação e HLS para entrega ao navegador. A avaliação considera uma matriz fatorial de quatro combinações tecnológicas submetidas a cargas de 1, 10 e 50 espectadores sintéticos, com três repetições por cenário. São analisados tempo de disponibilização da playlist HLS, consumo de CPU e memória, bitrate efetivo, taxa de erros HTTP e reinícios de sessão. Os resultados disponíveis indicam vantagem do SRS no startup da playlist e menor consumo de memória nos cenários com GStreamer, enquanto o GStreamer apresenta maior pressão de CPU sob carga. A latência ponta a ponta ainda requer instrumentação específica e, por isso, não é usada como evidência conclusiva nesta versão.

Palavras-chave: streaming de vídeo; RTMP; transcodificação; FFmpeg; GStreamer; desempenho; HLS.

Conteúdo

1	Introdução	3
1.1	Objetivo geral	3
1.2	Objetivos específicos	3
1.3	Contribuições	3
1.4	Organização do trabalho	4
2	Trabalhos Relacionados	4
2.1	Síntese da lacuna	4
3	Metodologia	5
3.1	Arquitetura e fluxo	5
3.2	Configuração do conteúdo e do pipeline	5
3.3	Métricas	6
3.4	Procedimento e análise	6
3.5	Validade e limitações	6
4	Resultados e Discussão	7
4.1	Síntese dos achados	7
4.2	O que ainda falta para uma conclusão forte	7
5	Conclusão	8
6	Referências	8

1 Introdução

O streaming ao vivo exige que uma cadeia de processamento receba o vídeo, transforme-o em representações adequadas e o entregue aos espectadores com continuidade e atraso reduzido. Nessa cadeia, o servidor de ingestão e o transcodificador podem consumir recursos de forma diferente e introduzir atrasos distintos. A escolha entre alternativas aparentemente equivalentes, portanto, deve ser apoiada por medições reproduzíveis, e não apenas por disponibilidade de documentação ou popularidade.

O projeto deste trabalho implementa uma plataforma de streaming composta por frontend, backend de controle, servidor RTMP, transcodificador e entrega HLS. O desenho permite alternar o servidor entre Nginx-RTMP e SRS e o transcodificador entre FFmpeg e GStreamer, mantendo a aplicação e o conteúdo de teste sob controle. Essa característica torna possível investigar o efeito da combinação dos componentes sobre uma mesma carga de trabalho.

O problema de pesquisa é: *como as combinações de servidores RTMP e transcodificadores influenciam o desempenho, o consumo de recursos e a estabilidade de uma plataforma de streaming de vídeo ao vivo em um ambiente containerizado?*

1.1 Objetivo geral

Avaliar o desempenho de servidores RTMP e transcodificadores no streaming de vídeo, comparando Nginx-RTMP e SRS combinados com FFmpeg e GStreamer em cargas controladas.

1.2 Objetivos específicos

- Descrever a arquitetura implementada e o fluxo RTMP–transcodificação–HLS.
- Definir uma matriz experimental que permita comparar servidor, transcodificador e número de espectadores.
- Medir tempo de disponibilização, CPU, memória, bitrate, erros de entrega e reinícios.
- Analisar a estabilidade dos cenários por médias e desvios padrão de três repetições.
- Identificar limitações da instrumentação e propor melhorias para uma avaliação posterior.

1.3 Contribuições

- Uma plataforma de streaming configurável e executável por Docker Compose.
- Um benchmark automatizado com dados brutos, logs e agregação estatística.
- Uma comparação experimental entre quatro combinações de software sob três cargas.
- Uma análise explícita das limitações de validade dos resultados disponíveis.

1.4 Organização do trabalho

O texto está organizado em seis seções principais. A Seção 2 apresenta os trabalhos relacionados; a Seção 3 descreve a metodologia; a Seção 4 apresenta e discute os resultados; a Seção 5 conclui o estudo e indica trabalhos futuros; e a Seção 6 reúne as referências utilizadas.

2 Trabalhos Relacionados

Estudos de desempenho em sistemas de streaming normalmente observam atraso, continuidade da reprodução, qualidade percebida, vazão e consumo de recursos. A literatura de streaming adaptativo destaca que o comportamento percebido pelo usuário resulta da interação entre qualidade de vídeo, variações de bitrate, tempo de início e interrupções [3, 4]. Neste trabalho, o foco é anterior à decisão adaptativa do cliente: avaliar como a ingestão RTMP e a geração das representações HLS afetam o comportamento do serviço.

Seufert et al. apresentam uma revisão de qualidade de experiência em HTTP Adaptive Streaming, discutindo fatores técnicos e perceptuais associados à adaptação de vídeo. Barman e Martini ampliam essa discussão ao revisar modelos de QoE e desafios de avaliação. Esses trabalhos orientam a seleção de métricas relacionadas à experiência, mas não comparam diretamente as quatro combinações implementadas neste projeto.

Em relação a protocolos, a especificação do RTMP descreve um protocolo de aplicação baseado em transporte confiável, adequado à multiplexação de áudio, vídeo e dados [1]. O HLS, por sua vez, organiza a apresentação em playlists e segmentos acessíveis por HTTP [2]. A solução avaliada usa RTMP na publicação e HLS na distribuição, refletindo uma arquitetura comum de ingestão e entrega desacopladas.

As documentações oficiais do FFmpeg e do GStreamer descrevem mecanismos diferentes de composição de pipelines [5, 6]. FFmpeg oferece uma ferramenta integrada para demux, filtros, codificação e mux; GStreamer organiza o processamento como uma cadeia de elementos conectados. A comparação deste trabalho não pretende declarar um transcodificador universalmente superior, pois os resultados dependem das versões, parâmetros, número de variantes e hardware utilizados.

A principal lacuna abordada é a comparação controlada entre servidor RTMP e transcodificador na mesma plataforma, com a mesma entrada, parâmetros de saída e cargas de espectadores. O uso de containers favorece a repetição do ambiente [9], enquanto Nginx-RTMP e SRS fornecem as alternativas de servidor avaliadas [7, 8]. O benchmark automatizado e os artefatos versionados aproximam o estudo de uma avaliação reprodutível, mas seus resultados permanecem limitados ao ambiente documentado.

2.1 Síntese da lacuna

Os trabalhos consultados oferecem fundamentos para protocolos, QoE e ferramentas multimídia. Contudo, não respondem diretamente qual combinação Nginx-RTMP/SRS e FFmpeg/GStreamer apresenta melhor comportamento sob as cargas definidas. A contribuição experimental deste TCC é preencher esse recorte com uma matriz comparável e explicitar as métricas que ainda não foram instrumentadas adequadamente.

3 Metodologia

Foi adotada uma abordagem quantitativa, experimental e comparativa. O fator servidor possui dois níveis (Nginx-RTMP e SRS), o fator transcodificador possui dois níveis (FFmpeg e GStreamer) e a carga possui três níveis (1, 10 e 50 espectadores sintéticos). O delineamento resulta em 12 tratamentos, cada um repetido três vezes, totalizando 36 execuções.

Tabela 1: Fatores e níveis do experimento

Fator	Níveis	Papel no experimento
Servidor RTMP	Nginx-RTMP; SRS	Recebe a publicação RTMP e dispara o processamento
Transcodificador	FFmpeg; GStreamer	Gera as variantes HLS
Carga	1; 10; 50 viewers	Simula demanda crescente de leitura
Repetição	3 por tratamento	Permite estimar média e desvio padrão

3.1 Arquitetura e fluxo

O streamer publica em RTMP. O servidor selecionado chama o callback do backend, que valida a chave da transmissão. Em seguida, o script de transcodificação lê o fluxo RTMP, codifica as variantes e escreve playlists e segmentos HLS. Viewers sintéticos requisitam a playlist e os segmentos pelo servidor HTTP do container. O backend, PostgreSQL, RabbitMQ e frontend fazem parte da plataforma, mas a medição de CPU e memória do benchmark tem como alvo principal o container RTMP/transcodificador.

Inserir o diagrama de arquitetura da plataforma nesta figura.

Figura 1: Arquitetura experimental da plataforma.

3.2 Configuração do conteúdo e do pipeline

O FFmpeg gera quatro variantes (1080p, 720p, 480p e 360p). O script atual do GStreamer também cria quatro playlists de variantes, embora versões anteriores da documentação mencionem duas; essa divergência deve ser eliminada antes da versão final. A duração nominal dos segmentos HLS é de 6 segundos, o comprimento da playlist é 10 e a codificação é realizada em CPU, sem GPU.

3.3 Métricas

Tabela 2: Métricas coletadas e interpretação

Métrica	Unidade	Definição operacional
Startup HLS	s	Tempo até a playlist mestre responder com sucesso
CPU média/máxima	%	Uso agregado do container RTMP durante a janela medida
Memória média/máxima	MB	Memória observada no container
Bitrate efetivo	kbps	Estimativa a partir dos segmentos HLS
Taxa de erros	%	Requisições HTTP com erro divididas pelo total
Reinícios	contagem	Diferença do contador de reinícios do container
Latência ponta a ponta	s	Campo previsto, mas não validado na coleta atual

3.4 Procedimento e análise

Cada cenário sobe uma instância limpa do ambiente containerizado, cria uma transmissão, publica uma fonte sintética, inicia a quantidade definida de viewers e coleta os dados durante aproximadamente cinco minutos. Os dados brutos são consolidados por uma rotina de agregação estatística, que calcula médias e desvios padrão agrupados por servidor, transcodificador e carga.

Os critérios operacionais atuais são startup HLS de até 24 s, CPU média de até 240% para 1 e 10 viewers e 280% para 50 viewers, taxa de erros de até 1% para 1 e 10 viewers e 3% para 50 viewers, além de zero reinícios. Esses limites são critérios de aceitação do experimento local, não requisitos universais de streaming.

3.5 Validade e limitações

Os resultados são internos ao host Ubuntu 24.04.3, Intel Core i5-13420H, 24 GB de RAM, SSD NVMe e Docker Engine. Não há rede externa, GPU, variação de codec, variação de resolução de entrada ou comparação com outros hosts. Além disso, a latência ponta a ponta está zerada ou não instrumentada na maior parte dos registros. A playlist mestre responder não significa necessariamente que o primeiro segmento válido já esteja disponível. Por esse motivo, essas métricas devem ser coletadas novamente com timestamps sincronizados entre publicação, criação do segmento, disponibilização HTTP e leitura pelo viewer.

4 Resultados e Discussão

Esta seção apresenta os resultados agregados das três repetições realizadas para cada tratamento experimental. Os valores reportados são médias; os desvios padrão foram calculados a partir das respectivas repetições. Os registros brutos foram preservados para auditoria e reprodutibilidade do experimento.

Tabela 3: Médias agregadas dos tratamentos

Servidor	Transcod.	Viewers	Startup (s)	CPU (%)	RAM (MB)	Bitrate (kbps)	Erros (%)
Nginx	FFmpeg	1	21,67	177,88	1658,06	3524,66	0,00
Nginx	FFmpeg	10	22,00	159,35	1679,64	3526,08	0,00
Nginx	FFmpeg	50	22,33	196,86	1640,75	3522,54	0,00
Nginx	GStreamer	1	22,00	233,44	657,02	5344,59	0,00
Nginx	GStreamer	10	22,00	243,48	623,47	5342,92	0,00
Nginx	GStreamer	50	22,00	292,13	629,52	5344,05	0,00
SRS	FFmpeg	1	8,00	195,32	1665,30	3155,57	0,00
SRS	FFmpeg	10	8,00	197,34	1678,86	3156,01	0,00
SRS	FFmpeg	50	8,00	215,74	1623,08	3157,79	0,00
SRS	GStreamer	1	1,00	207,80	642,26	5340,31	0,00
SRS	GStreamer	10	1,00	215,82	625,27	5338,16	0,00
SRS	GStreamer	50	1,00	192,35	500,90	5337,81	3,15

4.1 Síntese dos achados

Nos dados atuais, o SRS apresenta startup de playlist menor que o Nginx nos dois transcodificadores: 8 s contra aproximadamente 22 s com FFmpeg e 1 s contra 22 s com GStreamer. Essa vantagem deve ser descrita como vantagem de disponibilização da playlist, não como redução comprovada da latência percebida, pois a instrumentação de latência ponta a ponta é incompleta.

O FFmpeg consome aproximadamente 1,62–1,68 GB de memória média, enquanto o GStreamer fica aproximadamente entre 0,50 e 0,66 GB. Em contrapartida, o GStreamer apresenta maior CPU média, especialmente com Nginx: 243,48% em 10 viewers e 292,13% em 50 viewers. Portanto, os dados sugerem um trade-off entre memória e CPU, mas não permitem concluir que um transcodificador seja globalmente melhor sem normalizar o número de variantes, bitrate e parâmetros de codificação.

A taxa de erros foi zero em todos os tratamentos, exceto SRS + GStreamer + 50 viewers, que apresentou 3,15% e 1229 requisições com erro em média. Esse resultado é relevante porque ultrapassa o limite local de 3% e indica degradação sob carga alta. Deve-se investigar nos logs se os erros correspondem a segmentos ausentes, janela de playlist, saturação de conexão ou falha transitória do viewer.

4.2 O que ainda falta para uma conclusão forte

Antes da submissão final, recomenda-se repetir os 12 tratamentos após: (i) validar o mesmo número de variantes em FFmpeg e GStreamer; (ii) registrar versões exatas das imagens e ferramentas; (iii) medir latência ponta a ponta com um timestamp embutido no conteúdo; (iv) coletar percentis, especialmente p50, p95 e p99, para startup e requisições; e (v) separar erros de playlist, segmentos e conexão. Sem essas medidas, a conclusão deve

permanecer restrita a desempenho operacional do container e disponibilidade HLS.

5 Conclusão

Este trabalho propõe uma avaliação experimental de servidores RTMP e transcodificadores em uma plataforma de streaming de vídeo ao vivo. A implementação permite comparar Nginx-RTMP e SRS com FFmpeg e GStreamer sob 1, 10 e 50 viewers, com execução automatizada, coleta em CSV e agregação estatística.

Os resultados disponíveis sustentam três conclusões preliminares. Primeiro, o SRS apresentou menor tempo de disponibilização da playlist HLS nos cenários medidos. Segundo, o GStreamer apresentou menor consumo de memória média, mas maior pressão de CPU em parte das cargas. Terceiro, a combinação SRS + GStreamer apresentou taxa de erros acima do critério adotado com 50 viewers, indicando que a menor disponibilização inicial não implica necessariamente maior estabilidade sob carga.

Essas conclusões não devem ser generalizadas para qualquer hardware, codec ou configuração. O estudo foi executado em um único host, com codificação CPU-only, rede local e parâmetros específicos de resolução, bitrate e duração de segmento. A latência ponta a ponta também não está validada no conjunto atual. Assim, o principal resultado desta versão é uma base experimental reproduzível e uma comparação inicial, não um ranking universal das tecnologias.

Como trabalhos futuros, recomenda-se instrumentar a latência de ponta a ponta, incluir p95/p99, testar cargas superiores, repetir o experimento em outros hosts, avaliar aceleração por GPU, controlar rigorosamente as variantes de saída e investigar os erros de segmentos no tratamento SRS + GStreamer + 50 viewers. Após essas etapas, será possível formular uma conclusão mais forte sobre capacidade, escalabilidade e adequação operacional de cada combinação.

6 Referências

Referências

- [1] ADOBE SYSTEMS INCORPORATED. *Adobe's Real Time Messaging Protocol*. 2012. Disponível em: https://rtmp.veriskope.com/pdf/rtmp_specification_1.0.pdf. Acesso em: 11 ago. 2026.
- [2] PANTOS, R.; MAY, W. *HTTP Live Streaming*. RFC 8216. IETF, 2017. Disponível em: <https://www.rfc-editor.org/rfc/rfc8216>. Acesso em: 11 ago. 2026.
- [3] SEUFERT, M. et al. A survey on quality of experience of HTTP adaptive streaming. *IEEE Communications Surveys & Tutorials*, v. 17, n. 1, p. 469–492, 2015. DOI: <https://doi.org/10.1109/COMST.2014.2360940>.
- [4] BARMAN, N.; MARTINI, M. G. QoE modeling for HTTP adaptive video streaming – a survey and open challenges. *IEEE Access*, v. 7, p. 30831–30859, 2019. DOI: <https://doi.org/10.1109/ACCESS.2019.2901778>.

- [5] FFMPEG PROJECT. *FFmpeg Documentation*. Disponível em: <https://ffmpeg.org/documentation.html>. Acesso em: 11 ago. 2026.
- [6] GSTREAMER PROJECT. *GStreamer Documentation*. Disponível em: <https://gstreamer.freedesktop.org/documentation/>. Acesso em: 11 ago. 2026.
- [7] ARUT. *nginx-rtmp-module*. GitHub. Disponível em: <https://github.com/arut/nginx-rtmp-module>. Acesso em: 11 ago. 2026.
- [8] SRS PROJECT. *SRS: Simple Realtime Server Documentation*. Disponível em: <https://ossrs.io/lts/en-us/>. Acesso em: 11 ago. 2026.
- [9] DOCKER INC. *Docker Documentation*. Disponível em: <https://docs.docker.com/>. Acesso em: 11 ago. 2026.